

UNIVERSIDADE FEDERAL DE JUIZ DE FORA

DEPARTAMENTO DE CIÊNCIA DA COMPUTAÇÃO

DCC062 — SISTEMAS OPERACIONAIS — 2026.1

Android e Fuchsia

Trabalho Teórico — Tema 1

Pontos 1, 2 e 4

Autor

Felipe Lazzarini Cunha

Grupo 8

Juiz de Fora

Julho de 2026

Sumário

1	Apresentação sobre o tema	3
1.1	Visão geral	3
1.2	Android	3
1.3	Fuchsia	4
1.4	Por que comparar Android e Fuchsia	4
2	Propósito, evolução e virtualização	5
2.1	Propósito de cada sistema	5
2.2	Evolução no tempo	5
2.3	Mecanismos de virtualização	6
3	Tipos de aplicações atendidas pelo sistema operacional	7
3.1	Android	7
3.2	Fuchsia	8
4	Requisitos do tipo de aplicação atendido	8
4.1	Requisitos do Android	8
4.2	Requisitos do Fuchsia	9
4.3	Síntese comparativa dos requisitos	10
5	Atendimento dos requisitos pelos Sistemas Operacionais	11
5.1	Android	11
5.2	Fuchsia	12
6	Sistema de Interrupções do Kernel	13
6.1	Mecanismo e divisão de interrupções no Android	13
6.2	Gerenciamento de energia e propagação no Android	14
6.3	Arquitetura e tratamento de interrupções no Fuchsia	15
6.4	Ciclo de vida e isolamento de falhas no Zircon	16
7	Suporte a threads	16
7.1	Android	16
7.2	Fuchsia	17
8	Exclusão mútua, sincronização e IPC	17
8.1	Android	17
8.2	Fuchsia	18
9	Gerenciamento de Processos e escalonadores	18

9.1	Android	18
9.2	Fuchsia	19
10	Gerenciamento de Memória e Memória Virtual	19
10.1	Gerenciamento de memória e paginação no Android	19
10.2	Mecanismos de controle e otimização de RAM no Android	20
10.3	Abstração de memória no Fuchsia: VMOs e VMARs	20
10.4	Segurança por capacidades e isolamento de memória no Zircon	21
11	Sistema de entrada e saída e sistemas de arquivos	21
11.1	Android	21
11.2	Fuchsia	22
11.3	Comparação	22
12	Características Arquiteturais Distintivas	22
12.1	Cadeia de inicialização e o modelo Zygote no Android	22
12.2	Topologia de inicialização baseada em componentes no Fuchsia	23
12.3	Modularidade e a arquitetura Treble/APEX no Android	23
12.4	O paradigma de software efêmero no Fuchsia	24

1 Apresentação sobre o tema

1.1 Visão geral

O Tema 1 reúne dois sistemas operacionais desenvolvidos pela Google que, embora compartilhem a mesma mantenedora, representam abordagens arquiteturais profundamente distintas. De um lado, o **Android** é o sistema operacional móvel mais difundido do mundo, construído sobre um *kernel* Linux modificado e amplamente consolidado no mercado [2, 22]. De outro, o **Fuchsia** é um sistema operacional concebido “do zero” pela Google, estruturado sobre um *microkernel* próprio chamado Zircon e um modelo de segurança baseado em capacidades (*capability-based security*), que rompe com a tradição dos sistemas derivados do Unix [29, 54, 31].

Estudar os dois lado a lado é instrutivo justamente pelo contraste: o Android ilustra a evolução pragmática de um *kernel* monolítico maduro adaptado ao contexto móvel, enquanto o Fuchsia exemplifica uma tentativa de repensar, a partir de premissas modernas de segurança, modularidade e atualizabilidade, o que deve ser um sistema operacional de propósito geral [54, 39, 32]. As seções seguintes detalham cada sistema; aqui interessa situar o leitor quanto à origem, ao porte e ao estado atual de ambos.

1.2 Android

O Android teve origem na empresa *Android Inc.*, fundada em 2003 e adquirida pela Google em 2005. Sua primeira versão comercial chegou ao mercado em 2008, com o lançamento do aparelho HTC Dream [57]. Tecnicamente, trata-se de uma pilha de software cujo núcleo é um *kernel* Linux de suporte de longo prazo (LTS) acrescido de modificações específicas da Google — os chamados *Android Common Kernels* (ACKs) [13]. Grande parte dessa pilha é distribuída como software livre por meio do *Android Open Source Project* (AOSP) [2].

Em termos de adoção, o Android segue como o sistema operacional de maior alcance global quando se considera também o mercado móvel: estatísticas de uso na Web mantêm o Android na liderança entre sistemas operacionais móveis e também no agregado de todas as classes de dispositivo [55]. A versão estável consolidada mais recente no início desta pesquisa era o **Android 16**, lançado em 10 de junho de 2025; em junho de 2026, a documentação oficial da Google já apresentava o **Android 17**, ainda com notas de versão beta no ciclo consultado [23, 24]. O alcance do sistema vai de *smartphones* e *tablets* a televisores, automóveis, dispositivos vestíveis e experiências de tela grande baseadas na pilha Android, como o Googlebook [36].

1.3 Fuchsia

O Fuchsia veio a público de maneira atípica: em agosto de 2016, sem qualquer anúncio formal, jornalistas identificaram no GitHub um repositório de código mantido pela Google [58]. Diferentemente do Android, o Fuchsia não se baseia no *kernel* Linux. Seu núcleo — inicialmente denominado *Magenta* e renomeado para **Zircon** em setembro de 2017 — é um *microkernel* orientado a objetos com segurança baseada em capacidades, e o sistema é implementado em múltiplas linguagens, entre elas Rust, C++, C, Dart, Go e Python [58, 54].

O Fuchsia chegou a produtos de consumo pela primeira vez em maio de 2021, por meio de uma atualização de software do Google Nest Hub de primeira geração, que substituiu o sistema anterior (Cast OS, baseado em Linux); a implantação se estendeu aos demais aparelhos de primeira geração até agosto de 2021 e alcançou o Nest Hub de segunda geração em 2023 [58]. O projeto, contudo, enfrentou redução de equipe a partir de 2023 e, embora siga documentado e mantido em fontes oficiais recentes, seu papel estratégico tornou-se mais discreto do que se especulava em seu início [58, 29].

1.4 Por que comparar Android e Fuchsia

A comparação entre os dois sistemas é relevante por evidenciar duas filosofias opostas de construção de um sistema operacional. O Android adota um *kernel* monolítico (Linux), no qual grande parte dos serviços — *drivers*, pilha de rede, sistemas de arquivos — executa no espaço de *kernel*; já o Fuchsia adota um *microkernel* (Zircon) que mantém no núcleo apenas o essencial, delegando os demais serviços a componentes isolados em espaço de usuário e mediados por capacidades [54, 58]. Esse contraste sintetiza um debate clássico de Sistemas Operacionais entre desempenho e simplicidade do modelo monolítico, de um lado, e isolamento, segurança e atualizabilidade do modelo micronúcleo, de outro.

Por algum tempo especulou-se que o Fuchsia poderia vir a substituir ou unificar os sistemas da Google. O movimento recente da empresa, entretanto, aponta em outra direção: a documentação de desenvolvimento para **Googlebook** apresenta experiências de laptop apoiadas na pilha Android e em técnicas de aplicações adaptativas para telas grandes [36]. Assim, mais do que disputar o mesmo nicho, Android e Fuchsia ilustram caminhos tecnológicos distintos que a Google explorou em paralelo, o que torna o tema especialmente rico para um estudo de Sistemas Operacionais.

2 Propósito, evolução e virtualização

2.1 Propósito de cada sistema

O **Android** foi concebido como uma plataforma completa para dispositivos móveis e embarcados, especialmente aparelhos com tela sensível ao toque, conectividade permanente, bateria limitada e forte dependência de aplicativos de terceiros. Seu propósito não é apenas executar processos sobre um *kernel* Linux, mas oferecer uma pilha integrada que inclui *framework* de aplicação, tempo de execução gerenciado, serviços de mídia, segurança por aplicativo, distribuição de software e compatibilidade com grande diversidade de hardware. Por isso, o sistema combina um núcleo Linux adaptado ao ambiente móvel com mecanismos de isolamento, permissão e atualização voltados a um ecossistema amplo de fabricantes, operadoras, desenvolvedores e usuários finais [2, 13, 22].

O **Fuchsia**, por sua vez, tem um propósito mais experimental e arquitetural: servir como um sistema operacional de propósito geral, modular, atualizável e orientado à segurança desde a base. Em vez de herdar a arquitetura Unix/Linux, o Fuchsia é construído sobre o Zircon, um núcleo orientado a objetos que expõe recursos como capacidades; acima dele, quase todo o software é organizado como componentes isolados, com dependências e permissões declaradas explicitamente [54, 39, 31]. Essa proposta torna o Fuchsia particularmente interessante para dispositivos conectados e produtos nos quais isolamento, atualizabilidade e composição modular são requisitos centrais, ainda que sua presença comercial seja muito mais restrita que a do Android.

2.2 Evolução no tempo

A evolução do Android pode ser entendida em três eixos. O primeiro é a expansão de mercado: desde o Android 1.0, lançado comercialmente em 2008 no HTC Dream, o sistema deixou de ser uma plataforma de *smartphones* para se tornar uma família de variantes usadas também em televisores, automóveis, relógios, *tablets* e outros equipamentos [57, 22]. O segundo é a maturação técnica da pilha: as versões iniciais usavam a máquina virtual Dalvik, enquanto as versões modernas usam o Android Runtime (ART), que executa o formato DEX e introduziu compilação antecipada, melhorias de coleta de lixo e melhor suporte à depuração [3]. O terceiro eixo é a modularização do próprio sistema. Iniciativas como Treble separaram partes da plataforma Android das implementações específicas de fornecedores, enquanto o Mainline, introduzido no Android 10, passou a transformar componentes do sistema em módulos atualizáveis fora do ciclo tradicional de versões completas [59, 15].

No Fuchsia, a evolução foi menos orientada por versões públicas de produto e mais por

marcos arquiteturais. O projeto apareceu publicamente em 2016, inicialmente associado ao nome Magenta; em 2017, o núcleo passou a ser conhecido como Zircon [58, 54]. A primeira implantação em produto de consumo ocorreu em 2021, quando o sistema chegou ao Google Nest Hub de primeira geração, marcando a transição do projeto de uma base experimental para uso real em dispositivos Google [58]. Desde então, o desenvolvimento documentado enfatiza componentes, pacotes herméticos, interfaces FIDL, atualizações independentes e compatibilidade com software Linux por meio do Starnix [39, 32, 50].

Tabela 1: Marcos cronológicos de Android e Fuchsia.

Ano	Android	Fuchsia
2008	Primeira versão comercial, associada ao HTC Dream.	—
2014–2015	ART aparece como alternativa ao Dalvik e passa a substituí-lo nas versões modernas.	—
2016	—	Código do projeto aparece publicamente, ainda ligado ao nome Magenta.
2017	Treble reorganiza a relação entre plataforma Android e código de fornecedores.	O núcleo passa a ser conhecido como Zircon.
2019–2020	Android 10 introduz Mainline e módulos atualizáveis via APK/APEX.	Consolidação do modelo de componentes e capacidades.
2021	Android 12 amplia recursos de privacidade e isolamento no sistema.	Primeira implantação em produto de consumo, no Google Nest Hub.
2022–2026	AVF/pKVM se consolida como mecanismo moderno de virtualização protegida em dispositivos ARM64.	Starnix amadurece como camada de compatibilidade para executar programas Linux sem usar o kernel Linux.

2.3 Mecanismos de virtualização

No Android, é importante separar três ideias que às vezes aparecem misturadas sob o termo “virtualização”. A primeira é o **tempo de execução gerenciado**: Dalvik e ART não virtualizam hardware, mas fornecem uma máquina de execução para *bytecode* DEX, permitindo que aplicativos escritos em Java ou Kotlin sejam executados de forma portátil sobre diferentes arquiteturas de CPU [3]. A segunda é o **isolamento por aplicativo**: cada app executa em um contexto separado, com identificador, permissões e políticas de segurança próprias; isso é uma forma de compartimentalização do sistema, embora não seja uma máquina virtual completa.

A virtualização de hardware propriamente dita aparece no Android moderno por meio

do **Android Virtualization Framework** (AVF). Segundo a documentação do AOSP, o AVF fornece ambientes de execução seguros e privados para código que exige isolamento mais forte que o *sandbox* tradicional de aplicativos. Em dispositivos ARM64 compatíveis, o AVF usa o pKVM (*protected kernel-based virtual machine*) como hipervisor e executa *protected virtual machines* (pVMs), que podem hospedar, por exemplo, o Microdroid, uma distribuição Android mínima voltada à execução isolada [5]. O ponto central desse modelo é proteger a memória e os segredos da máquina virtual mesmo diante de comprometimento do Android hospedeiro.

No Fuchsia, a arquitetura básica favorece isolamento por desenho, e não por uma camada de virtualização semelhante ao AVF. O Zircon fornece primitivas de processos, *threads*, memória virtual e IPC; acima dele, o *Component Framework* executa componentes em *sandboxes* com capacidades declaradas explicitamente [54, 39, 31]. Assim, a unidade normal de isolamento do sistema é o componente com capacidades, não uma VM.

Para compatibilidade com software Linux, a documentação atual do Fuchsia destaca o **Starnix**. Ele permite executar programas Linux não modificados em um sistema Fuchsia, implementando a interface de chamadas de sistema Linux em um programa de espaço de usuário do Fuchsia, em vez de inicializar um kernel Linux convidado [50, 51]. Portanto, para o escopo deste trabalho, a comparação mais precisa é: o Android possui virtualização protegida de hardware com AVF/pKVM, enquanto o Fuchsia aposta principalmente em isolamento por capacidades e componentes, usando Starnix como camada de compatibilidade Linux, e não como hipervisor equivalente.

3 Tipos de aplicações atendidas pelo sistema operacional

3.1 Android

O Android foi desenvolvido principalmente para executar aplicações **interativas**, sendo utilizado em smartphones, tablets, televisores, relógios inteligentes, sistemas automotivos e diversos dispositivos embarcados. Seu foco está na interação direta com o usuário por meio de interfaces gráficas, com requisitos elevados de responsividade e suporte à execução simultânea de múltiplas aplicações.

Além das aplicações interativas, o Android oferece suporte à execução de serviços em segundo plano responsáveis por sincronização de dados, notificações, processamento periódico, reprodução de mídia e comunicação em rede. Entretanto, o sistema não é classificado como um sistema operacional de tempo real (RTOS), embora seja capaz de executar aplicações que exijam baixa latência, como áudio e vídeo, graças às otimizações presentes no kernel

Linux e na plataforma Android [2, 13].

O Android utiliza uma versão modificada do kernel Linux, cuja arquitetura é classificada como *monolítica modular*. Nesse modelo, serviços essenciais como gerenciamento de memória, processos, dispositivos de entrada e saída, drivers e sistemas de arquivos executam em espaço de kernel, enquanto módulos permitem ampliar o suporte a diferentes dispositivos de hardware sem alterar o núcleo principal do sistema [13].

3.2 Fuchsia

O Fuchsia foi projetado para atender uma ampla variedade de dispositivos, incluindo equipamentos embarcados, dispositivos inteligentes, computadores e outros sistemas conectados. Assim como o Android, seu principal foco está em aplicações interativas, porém sua arquitetura prioriza modularidade, escalabilidade, isolamento e segurança.

O sistema também pode ser empregado em aplicações que demandam baixa latência e alta confiabilidade, embora não seja considerado um sistema operacional de tempo real tradicional. Sua arquitetura foi desenvolvida para fornecer maior previsibilidade e isolamento entre componentes quando comparada aos sistemas baseados em kernels monolíticos [29].

O Fuchsia utiliza o kernel Zircon, desenvolvido especificamente para esse sistema operacional. O Zircon apresenta uma arquitetura *microkernel-like*, mantendo no núcleo apenas os mecanismos essenciais de gerenciamento de processos, threads, memória virtual, comunicação entre processos e escalonamento, enquanto diversos serviços, como drivers e sistemas de arquivos, executam em espaço de usuário [54].

4 Requisitos do tipo de aplicação atendido

Este ponto identifica *quais* requisitos as aplicações atendidas por cada sistema impõem ao SO/kernel. Na terminologia clássica de Sistemas Operacionais, um baixo *tempo de resposta* é próprio de cargas interativas, ao passo que um baixo *tempo de retorno* (*turnaround*) e a alta vazão caracterizam cargas em lote, e a *previsibilidade* temporal distingue cargas de tempo real [56]. A classificação do tipo de aplicação (Ponto 3) e a forma como esses requisitos são efetivamente atendidos (Ponto 5) cabem a outros integrantes; aqui o foco recai sobre *quais* requisitos cada sistema precisa satisfazer.

4.1 Requisitos do Android

As aplicações atendidas pelo Android são majoritariamente **interativas** e executam em dispositivos móveis alimentados por bateria, com recursos de hardware limitados e forte

presença de software de terceiros. Esse perfil impõe ao sistema um conjunto bem definido de requisitos:

- **Baixo tempo de resposta (responsividade).** A interface deve reagir à entrada do usuário quase instantaneamente e manter as animações fluidas. Na prática, cada quadro precisa ser produzido dentro do orçamento de tempo do ciclo de atualização da tela — cerca de 16,7 ms a 60 Hz e aproximadamente 8,3 ms a 120 Hz —, sob pena de *jank* (perda de quadros) perceptível. O sistema trata a responsividade como requisito explícito, com sincronização de quadros (VSync, *frame pacing*) e o monitoramento de *Application Not Responding* (ANR), que sinaliza falha quando um aplicativo deixa de responder a um evento de entrada por alguns segundos [16, 3].
- **Eficiência energética.** Como a autonomia de bateria é finita, minimizar o consumo de energia é um requisito de primeira ordem: o sistema precisa conter o gasto de processos em segundo plano e respeitar limites térmicos, apoiando-se em mecanismos como Doze, hibernação de aplicativos e mitigação térmica [16].
- **Operação sob restrição de memória e recursos.** Dispositivos móveis dispõem de pouca memória e de pouca ou nenhuma área de troca convencional; o sistema deve permanecer estável sob pressão de memória, o que exige uso parcimonioso de RAM pelos aplicativos e uma política ativa de recuperação de memória, exercida em espaço de usuário pelo *low memory killer daemon* (lmkd) [16].
- **Segurança e isolamento por aplicativo.** Por reunir aplicativos de muitas origens, o Android precisa isolar cada aplicativo dos demais e do sistema, protegendo os dados do usuário. Esse requisito aparece tanto no *sandbox* padrão de cada aplicativo quanto na oferta de ambientes de execução reforçados para cargas mais sensíveis, providos pelo AVF [2, 5].
- **Portabilidade e compatibilidade de hardware.** Por fim, atender um ecossistema com enorme diversidade de aparelhos torna a portabilidade e a compatibilidade requisitos estruturais, refletidos na separação entre a plataforma e o código dos fornecedores [13, 59].

4.2 Requisitos do Fuchsia

O Fuchsia também visa dispositivos conectados e produtos de propósito geral, mas hierarquiza os requisitos de modo distinto: em vez de otimizar primeiro a experiência interativa imediata, eleva à condição de requisito atributos estruturais do sistema. A própria documentação organiza o projeto em torno de princípios de design, dos quais se destacam, para este ponto, a segurança e a atualizabilidade [31, 32].

- **Segurança por capacidades.** O sistema parte do princípio de menor privilégio e da ausência de autoridade ambiente: um componente só interage com os objetos cujas capacidades lhe foram concedidas explicitamente, e tudo executa em *sandbox*, com namespace próprio em vez de um sistema de arquivos global [31, 54].
- **Atualizabilidade.** O software é entregue em pacotes atualizáveis de forma independente e sob demanda, à semelhança da Web, de modo que uma correção de vulnerabilidade possa alcançar todos os produtos sem coordenação individual por dispositivo [32].
- **Modularidade e composição.** O sistema é montado a partir de componentes isolados, com dependências e capacidades declaradas explicitamente; a composição modular é, assim, um requisito de projeto, e não um mero detalhe de implementação [39].
- **Escalabilidade entre classes de dispositivo.** O Zircon e a pilha de componentes foram concebidos para abranger desde dispositivos pequenos até equipamentos mais capazes, o que faz da escalabilidade um requisito presente desde a base [29, 54].
- **Compatibilidade com o ecossistema Linux.** Para ser viável na prática, o sistema precisa executar software já existente; esse requisito de compatibilidade é atendido pelo Starnix, que implementa a interface de chamadas de sistema do Linux sobre o próprio Fuchsia [50].

4.3 Síntese comparativa dos requisitos

Embora ambos atendam, em última instância, aplicações interativas e conectadas, Android e Fuchsia hierarquizam seus requisitos de maneiras diferentes. O Android prioriza requisitos percebidos diretamente pelo usuário final — responsividade e autonomia de bateria —, enquanto o Fuchsia coloca em primeiro plano requisitos estruturais do sistema — segurança por capacidades e atualizabilidade. A Tabela 2 resume esse contraste.

Em síntese, os requisitos do Android giram em torno de *tempo de resposta curto e previsível, economia de energia e operação sob recursos limitados*, somados ao isolamento por aplicativo; os do Fuchsia giram em torno de *segurança por capacidades, atualizabilidade e modularidade*, tendo a escalabilidade e a compatibilidade como requisitos de viabilidade. O modo como cada sistema satisfaz esses requisitos é tratado no Ponto 5.

Tabela 2: Requisitos prioritários de Android e Fuchsia.

Requisito	Android	Fuchsia
Tempo de resposta	Crítico: interface interativa dentro do orçamento de quadro (60–120 Hz).	Relevante, mas não é o eixo prioritário do projeto.
Eficiência energética	Crítica: autonomia de bateria no dispositivo móvel.	Importante conforme a classe de dispositivo.
Memória e recursos	Operação sob forte restrição, com recuperação ativa (lmkd).	Custo de isolamento previsto no modelo de componentes.
Segurança e isolamento	Por aplicativo (sandbox e UID; AVF para cargas sensíveis).	Por capacidades, no núcleo do projeto (menor privilégio).
Atualizabilidade	Modular e incremental (Treble, Mainline).	Princípio central: pacotes atualizáveis sob demanda.
Escalabilidade de hardware	Ampla, consolidada pela adaptação do ecossistema.	Prevista por desenho desde o Zircon.

5 Atendimento dos requisitos pelos Sistemas Operacionais

5.1 Android

No Android, a responsividade depende de um caminho bem definido entre a entrada do hardware, a *thread* principal da aplicação e a composição final no display. Ao iniciar um aplicativo, o sistema cria um processo Linux com uma *thread* principal, na qual executam os componentes padrão da aplicação e o tratamento de eventos da interface gráfica. Essa *thread* é organizada em torno de um **Looper**, uma **MessageQueue** e objetos **Handler**; por isso, operações longas nela bloqueiam diretamente a interface e prejudicam o tempo de resposta percebido pelo usuário [47].

A entrada do usuário é normalizada antes de chegar ao aplicativo. Os *drivers* traduzem sinais de telas sensíveis ao toque, botões, teclados e outros dispositivos em eventos do Linux; em seguida, o **InputReader** interpreta esses eventos no modelo do Android, e o **InputDispatcher** os encaminha à janela apropriada [12]. A sincronização dos quadros fica a cargo do **Choreographer**, que coordena o ritmo de animações, entrada e desenho, enquanto a composição final usa **VSync**, **SurfaceFlinger** e **Hardware Composer** para reduzir perdas de quadro e manter a fluidez gráfica [27, 2]. Quando esse contrato falha, o sistema detecta situações de *Application Not Responding* (ANR), tipicamente associadas a um evento de entrada sem resposta por cerca de 5 segundos; serviços em primeiro e segundo plano também possuem limites próprios de tempo de resposta [28].

O requisito de eficiência energética é tratado pelo Android por meio de restrições explícitas à execução ociosa e em segundo plano. O sistema suspende o dispositivo sempre que possível, limita tarefas de fundo, aplica mitigação térmica e usa mecanismos como Doze para adiar trabalho de CPU quando o aparelho permanece inativo por tempo prolongado, abrindo janelas periódicas para manutenção de atividades pendentes [17, 16]. O `SystemSuspend` mantém o controle da suspensão do dispositivo e só permite que ela ocorra quando não há *wakelocks* impedindo a entrada em estado de baixo consumo [19]. Além disso, recursos como App Standby Buckets, hibernação de aplicativos e sinais térmicos ajudam a graduar restrições conforme o uso recente e o estado físico do aparelho [42, 6, 20].

Sob pressão de memória, o Android combina mecanismos herdados do Linux com políticas próprias voltadas à experiência interativa. O *kernel* usa compressão, recuperação de páginas e o `kswapd`; quando isso não basta, a plataforma prioriza processos e encerra os menos importantes para preservar o aplicativo em primeiro plano e serviços essenciais [43]. Nas versões modernas, essa decisão é coordenada em espaço de usuário pelo `lmkd`, que acompanha a pressão de memória e aciona encerramentos conforme a importância do processo para o usuário [14].

Em segurança, o Android atende aos requisitos de isolamento combinando o *sandbox* por aplicativo do Linux, UIDs distintos, permissões, SELinux, assinatura de aplicativos, *verified boot* e IPC controlado. Cada aplicativo executa em seu próprio processo e com seu próprio UID, o que restringe o acesso a dados e recursos de outros aplicativos [7, 18]. A integridade da base do sistema é reforçada pelo Android Verified Boot, que verifica a cadeia de inicialização e dificulta a persistência de alterações não autorizadas no software do dispositivo [4]. Para comunicação entre processos, o Android usa o Binder IPC, que abstrai chamadas a serviços remotos e serve tanto à comunicação entre aplicativos quanto à comunicação entre aplicativos e serviços do sistema [10].

Por fim, a portabilidade é atendida por uma arquitetura que separa a plataforma Android dos serviços específicos de hardware. As HALs oferecem interfaces padronizadas implementadas pelos fabricantes, reduzindo o acoplamento entre *framework*, *drivers* e dispositivos concretos [8]. No nível do *kernel*, o projeto Generic Kernel Image (GKI) estabiliza interfaces entre o núcleo comum e módulos de fornecedores, permitindo atualizar partes do sistema com maior independência [11].

5.2 Fuchsia

No Fuchsia, o atendimento dos requisitos parte de uma base arquitetural distinta: o Zircon é um núcleo orientado a objetos e baseado em capacidades. Processos são isolados por padrão, e recursos do *kernel* são acessados por *handles*, isto é, referências locais a objetos com direitos específicos associados às operações permitidas [31, 40]. Um novo processo começa

sem autoridade própria relevante e só recebe recursos que lhe são explicitamente passados pelo criador; como canais de IPC também podem transferir *handles*, o sistema consegue mover tanto dados quanto capacidades entre componentes de forma controlada [48, 40].

A atualizabilidade é tratada como requisito estrutural. Em vez de organizar todo o software como uma aplicação monolítica ou uma imagem rígida de sistema, o Fuchsia distribui software em pacotes que podem ser atualizados de forma independente e obtidos sob demanda [44, 32]. O versionamento da plataforma busca permitir evolução sem quebrar componentes antigos: o Component Manager pode impedir a inicialização de componentes cuja compatibilidade com o produto não esteja garantida pelas regras documentadas da plataforma [45].

A modularidade decorre do modelo de componentes. As unidades principais de software executam em seus próprios ambientes, comunicam-se por capacidades e declaram suas necessidades em manifestos. Cada componente é identificado por URL e descrito por um manifesto que especifica execução e capacidades [39, 38]. A topologia também é explícita: componentes pais oferecem capacidades a componentes filhos, o que torna a dependência entre partes do sistema visível e controlável [30].

Para escalar entre classes de dispositivos, o Fuchsia usa configurações de *build* orientadas a produto. Essas configurações descrevem arquitetura, hardware, *drivers* e outros parâmetros, orientando quais componentes e capacidades entram em cada imagem gerada [25]. Já a compatibilidade com o ecossistema Linux é buscada por meio do Starnix, um *runner* capaz de criar um ambiente Linux em espaço de usuário. Essa solução permite executar algumas aplicações Linux sem modificação, mas não transforma o Fuchsia em um sistema totalmente compatível com Linux; a compatibilidade depende do comportamento esperado por cada aplicação [50].

6 Sistema de Interrupções do Kernel

6.1 Mecanismo e divisão de interrupções no Android

O sistema de interrupções no Android, no nível do *kernel*, opera de forma estruturalmente semelhante ao de um sistema Linux tradicional. No entanto, o Android introduz otimizações agressivas e mecanismos específicos — como o *driver* Binder e o gerenciamento de memória focado em *proportional set size* e *low memory killer* — voltados para arquiteturas móveis (ARM/ARM64) e para o controle estrito de energia. O processamento de uma interrupção ocorre em camadas, iniciando-se no estímulo físico e culminando na resposta do sistema operacional.

Quando um evento físico ocorre, como um toque na tela, o controlador de hardware gera

um sinal elétrico denominado requisição de interrupção (IRQ — *Interrupt Request*). Nos dispositivos Android modernos, esse sinal é interceptado pelo Controlador Genérico de Interrupções (GIC — *Generic Interrupt Controller*). O GIC atua como um gerenciador central: ele prioriza as interrupções, determina qual núcleo do processador deve tratá-las (afinidade de IRQ) e sinaliza a CPU para alternar seu modo de operação. Nesse momento, a CPU suspende a instrução corrente, salva o contexto atual e desvia o fluxo de execução para um endereço de memória predefinido, conhecido como vetor de interrupção (e seu respectivo IDT). A partir desse ponto, o *kernel* assume o controle por meio do manipulador de interrupções principal (*Interrupt Handler*).

Para evitar o congelamento do sistema, o processamento da interrupção não ocorre de forma integral e síncrona. Se o *kernel* executasse tarefas complexas, como a leitura detalhada de dados de toque e a renderização gráfica, com as interrupções desabilitadas, o sistema apresentaria falhas de responsividade, como engasgos no áudio. Assim, a arquitetura divide o trabalho em duas etapas fundamentais: o *Top Half* e o *Bottom Half*. O *Top Half* (ou *Hard IRQ / Fast Handler*) é o código associado diretamente à linha da IRQ e concebido para ser executado com extrema rapidez. Sua função restringe-se a reconhecer a interrupção perante o GIC, limpar o registrador de hardware, salvar os dados brutos em um *buffer* rápido e agendar a execução do estágio seguinte. Durante essa fase, outras interrupções locais permanecem desabilitadas.

Em seguida, o *Bottom Half* (ou *Slow Handler*) é executado com as interrupções de hardware reabilitadas, permitindo a recepção de novos eventos. No ecossistema Android/Linux, essa etapa utiliza mecanismos distintos. Os *SoftIRQs* são reservados para tarefas críticas de subsistemas que não podem entrar em estado de espera (como temporizadores e redes). Os *Tasklets* são construídos sobre os *SoftIRQs* para facilitar o uso por desenvolvedores de *drivers*. Por fim, as *Workqueues* executam no contexto de *threads* do *kernel* (**kworker**) e permitem a suspensão da tarefa (estado de *sleep*), sendo o ambiente onde a maior parte do processamento de *drivers* complexos do Android é realizada.

6.2 Gerenciamento de energia e propagação no Android

Uma distinção crítica do Android em relação aos sistemas convencionais reside no tratamento de energia. Quando inativa, a tela é bloqueada e a CPU entra em estados de suspensão profunda (*deep sleep*) para preservar a bateria. Nesse contexto, nem toda interrupção possui permissão para reativar o processador. Interrupções de rotina, como o recebimento de pacotes de rede não essenciais pelo Wi-Fi, não alteram o estado de dormência do sistema.

Para eventos que exigem processamento imediato — como o acionamento do botão de energia, a detecção de um toque duplo pelo acelerômetro ou o recebimento de uma chamada

telefônica — utilizam-se as interrupções de despertar (*Wakeup IRQs*). Estas são configuradas explicitamente nos *drivers* por meio de chamadas como `enable_irq_wake()`. Ao detectá-las, o GIC instrui o circuito de gerenciamento de energia (PMIC) a religar os cores da CPU. Uma vez reativado o *kernel*, o sistema recorre a *Wakelocks* (atualmente integrados ao mecanismo de *autosleep* do Linux). O *kernel* adquire esse bloqueio enquanto processa a interrupção no *Bottom Half*, garantindo que o hardware não retorne prematuramente ao estado de suspensão antes da conclusão da tarefa.

Após o processamento primário pelo *kernel*, o evento precisa ser roteado até o aplicativo no espaço de usuário. Inicialmente, o *driver* de entrada encaminha os dados processados para o subsistema correspondente do *kernel*, disponibilizando-os em interfaces padronizadas, como os arquivos `/dev/input/eventX` (subsistema *evdev*). No *framework* do Android, uma *thread* nativa escrita em C++, denominada **InputReader**, monitora continuamente esses arquivos de eventos. Ao registrar novos dados, ela os repassa ao **InputDispatcher**, que, por sua vez, utiliza a comunicação interprocessos (IPC) via Android Binder ou *Unix Domain Sockets* para entregar a informação ao processo do aplicativo que possui o foco atual na tela. Por fim, o aplicativo intercepta a mensagem em sua *thread* principal (*UI Thread* via *Looper/Handler*), acionando manipuladores de código específicos, como o método `onTouchEvent()`, e completando o ciclo de interação.

6.3 Arquitetura e tratamento de interrupções no Fuchsia

Diferentemente do modelo adotado pelo *kernel* Linux no Android, no qual os *drivers* de dispositivos e o processamento de interrupções compartilham o mesmo espaço de endereçamento do núcleo, o Fuchsia transfere a execução dos *drivers* para o espaço de usuário (*user space*). Essa abordagem arquitetural modifica profundamente o gerenciamento de interrupções no sistema.

Enquanto o Android divide o tratamento em duas metades complexas para mitigar a latência do núcleo, o Zircon elimina o conceito de *Bottom Half* de dentro do ambiente do *kernel*. O micronúcleo do Fuchsia executa apenas o mínimo absoluto necessário para interagir com o hardware antes de delegar a continuidade do fluxo. O papel do Zircon restringe-se a interceptar o sinal elétrico de hardware (IRQ), mascarar temporariamente aquela interrupção específica no controlador (como o GIC) para evitar a saturação da CPU, sinalizar a camada superior sobre a ocorrência do evento e devolver imediatamente o processador para a execução das tarefas anteriores.

Sendo um sistema operacional orientado a objetos e estruturado sob o princípio de segurança baseada em capacidades, o Fuchsia mapeia as linhas físicas de interrupção de hardware por meio de uma abstração denominada *Interrupt Object*. Para que um *driver* — que opera como um processo comum em espaço de usuário — possa interagir com periféricos como

placas de rede ou superfícies sensíveis ao toque, é indispensável a posse de um *Handle* (um token de permissão) válido e intransferível associado ao respectivo *Interrupt Object*. A ausência desse descritor impede estritamente a visibilidade ou a manipulação do hardware, assegurando o isolamento completo entre os componentes.

6.4 Ciclo de vida e isolamento de falhas no Zircon

Devido à descentralização dos *drivers* para o espaço de usuário, a gerência do ciclo de vida das interrupções ocorre por meio de chamadas de sistema (*syscalls*). O fluxo operacional inicia-se com o estado de espera, no qual uma *thread* do *driver* em espaço de usuário invoca a *syscall* `zx_interrupt_wait()`. Essa chamada bloqueia a *thread*, colocando-a em estado de dormência até que o evento físico se manifeste.

No momento em que o hardware gera a IRQ, o Zircon intercepta o sinal, efetua o mascaramento no nível do controlador e acorda a *thread* correspondente que aguardava naquele *Interrupt Object*. Ao retomar a execução no espaço de usuário, o *driver* assume o controle e realiza o processamento pesado dos dados, efetuando leituras de barramentos, decodificação de pacotes de rede ou interpretação de coordenadas de toque. Concluída a manipulação dos dados, o *driver* invoca a *syscall* `zx_interrupt_ack()`. O Zircon então desmascara a IRQ no hardware, restabelecendo a recepção de novos sinais vindos daquele dispositivo, enquanto o *driver* retorna à chamada de espera, reiniciando o ciclo.

O benefício central dessa engenharia reside no isolamento de falhas do sistema operacional. No modelo monolítico do Android, uma falha de ponteiro ou uma violação de acesso cometida por um *driver* durante o processamento no *Bottom Half* resulta inevitavelmente em um pânico de núcleo (*Kernel Panic*), o que compromete a integridade de todo o sistema e força a reinicialização do dispositivo. No Fuchsia, caso ocorra uma falha análoga durante o ciclo de processamento da interrupção, o impacto restringe-se exclusivamente ao componente isolado do *driver* em espaço de usuário. O núcleo Zircon permanece preservado e operacional, permitindo que o gerenciador de componentes do Fuchsia reinicie o *driver* afetado de forma transparente e em poucos milissegundos.

7 Suporte a threads

7.1 Android

O Android possui suporte nativo à execução concorrente por meio de **threads de nível de kernel** (Kernel-Level Threads), implementadas pelo próprio kernel Linux. Cada aplicação é executada como um processo Linux independente e inicia sua execução com uma thread principal, responsável pelo processamento da interface gráfica da aplicação.

Sempre que necessário, novas threads podem ser criadas utilizando as APIs da plataforma, como a classe `Thread`, mecanismos baseados em `Executor` ou corrotinas da linguagem Kotlin. Independentemente da API utilizada, o escalonamento das threads é realizado pelo kernel Linux, que compartilha entre elas os recursos pertencentes ao mesmo processo, como memória e descritores de arquivos [47].

7.2 Fuchsia

No Fuchsia, o suporte à concorrência também é implementado por meio de threads de nível de kernel. Entretanto, diferentemente do Linux, cada thread é tratada como um objeto do próprio kernel Zircon.

As threads podem ser criadas, iniciadas, suspensas e encerradas utilizando as chamadas de sistema disponibilizadas pelo Zircon. O escalonador do kernel é responsável por controlar prioridades, sincronização e distribuição das threads entre os núcleos do processador, oferecendo boa responsividade para aplicações interativas e elevado isolamento entre componentes do sistema [52].

8 Exclusão mútua, sincronização e IPC

8.1 Android

No Android, a exclusão mútua começa no próprio modelo de isolamento: cada aplicativo iniciado recebe um processo Linux próprio, com uma *thread* principal e a possibilidade de criar *threads* auxiliares. Esse isolamento entre processos protege recursos de aplicativos distintos, mas o estado compartilhado dentro de um mesmo processo continua exigindo mecanismos tradicionais de segurança de *threads* [47]. Na camada de aplicação, essas garantias são normalmente implementadas com os recursos de sincronização oferecidos por Java e Kotlin; em Java, por exemplo, os métodos `wait()`, `notify()` e `notifyAll()` fazem parte do modelo de monitores associado a `Object` [41].

A sincronização da interface é organizada pela *thread* principal. O `Looper` executa o laço de mensagens da *thread*, a `MessageQueue` mantém as mensagens e tarefas pendentes, e os `Handlers` publicam mensagens ou objetos `Runnable` nessa fila [47]. Esse confinamento é um padrão importante: quando o estado da interface é manipulado apenas pela *thread* principal, muitos *locks* explícitos deixam de ser necessários. Para operações longas ou bloqueantes, o padrão recomendado é deslocar trabalho para *worker threads*; em Kotlin, co-rotinas fornecem uma forma usual de expressar trabalho assíncrono sem bloquear a interface [37].

Como processos Android não acessam diretamente a memória uns dos outros, a comunicação

entre processos usa IPC. O mecanismo central é o Binder, estruturado como uma arquitetura cliente-servidor na qual um processo invoca operações expostas por outro e recebe respostas mediadas por um *driver* do *kernel* Linux [10, 9]. O Service Manager funciona como diretório de serviços do sistema, permitindo que clientes localizem objetos remotos, enquanto a Android Interface Definition Language (AIDL) oferece uma forma de declarar interfaces de alto nível e gerar o código de serialização das mensagens [9].

8.2 Fuchsia

No Fuchsia, sincronização e IPC são construídos sobre objetos do Zircon. A interação do espaço de usuário com recursos do *kernel* ocorre por meio de *handles*, e o *kernel* verifica a validade e os direitos desses *handles* em cada operação [53]. Para comunicação, o sistema oferece canais bidirecionais, que transmitem mensagens e podem transferir *handles*, e *sockets*, usados para fluxos de bytes [26, 49].

O Fuchsia também fornece futexes no Zircon como primitiva de baixo nível para construir mecanismos de sincronização de *threads*. Esses futexes diferem do modelo Linux porque são locais ao processo: eles servem para sincronizar *threads* dentro do mesmo processo, e não para coordenar diretamente processos não relacionados [35]. Essa escolha é coerente com o modelo geral do Fuchsia, no qual a sincronização interna fica contida no processo e a coordenação entre componentes ocorre por canais, capacidades e protocolos explícitos.

9 Gerenciamento de Processos e escalonadores

9.1 Android

No Android, o processo raiz para criação de aplicativos é o Zygote. Durante a inicialização, esse processo carrega o Android Runtime e partes centrais do *framework*; quando um novo aplicativo precisa ser iniciado, o sistema solicita ao Zygote, por meio de um *Unix domain socket*, a criação de um novo processo por `fork()` [21]. O uso de *copy-on-write* permite que páginas comuns permaneçam compartilhadas até que precisem ser modificadas, reduzindo o custo de inicialização e o consumo de memória.

Como o Android é construído sobre o *kernel* Linux, o escalonamento de CPU é realizado, em última instância, pelo próprio Linux, ainda que a plataforma adicione políticas específicas para preservar a fluidez da experiência móvel. O sistema classifica processos conforme sua importância para o usuário e usa essa hierarquia tanto para priorizar recursos quanto para decidir quais processos podem ser encerrados quando há pressão de memória [46, 43]. A Tabela 3 resume essa hierarquia.

Tabela 3: Hierarquia simplificada de prioridade de processos no Android.

Tipo de processo	Nível de prioridade
Primeiro plano	Máximo: processo diretamente necessário para a interação atual.
Processo visível	Muito alto: afeta algo visível ao usuário, ainda que não esteja em foco.
Serviço	Médio: executa trabalho perceptível ou solicitado por componentes do sistema.
Segundo plano	Baixo: candidato a encerramento quando há necessidade de memória.
Processo vazio	Muito baixo: mantido apenas como <i>cache</i> e encerrado prioritariamente.

9.2 Fuchsia

No Fuchsia, o gerenciamento de aplicações e o escalonamento de CPU são conceitualmente separados. O Zircon gerencia objetos de baixo nível, como processos e *threads*, mas não trata aplicações como unidades semânticas centrais; essa função pertence ao modelo de componentes e aos *runners* [53, 39]. Assim, o mapeamento entre um componente e um processo depende das políticas do componente e do *runner* usado para executá-lo.

Entre os *runners* relevantes estão o ELF Runner, usado para binários nativos compilados de linguagens como C, C++ ou Rust, o Web Runner, voltado a aplicações Web, e o Starnix, que executa programas Linux em espaço de usuário [39, 50]. Essa organização reforça a separação entre o que o *kernel* precisa escalonar, isto é, *threads*, e o que a plataforma precisa administrar como software de produto, isto é, componentes com capacidades, manifestos e ciclos de vida próprios.

10 Gerenciamento de Memória e Memória Virtual

10.1 Gerenciamento de memória e paginação no Android

O gerenciamento de memória no Android baseia-se diretamente no subsistema de gerenciamento de memória do *kernel* Linux, extensamente adaptado para lidar com as severas restrições de hardware, ciclos de bateria e ausência de armazenamento secundário persistente convencional em dispositivos móveis. O sistema utiliza uma arquitetura de memória virtual puramente baseada em paginação sob demanda, mapeando o espaço de endereçamento lógico dos processos para as páginas físicas da memória RAM por meio da Unidade de Gerenciamento de Memória (MMU). No Android, cada aplicativo executa dentro de seu próprio espaço de endereçamento virtual isolado de 64 bits (em plataformas ARM64 modernas), mitigando interferências mútuas e implementando técnicas avançadas de segurança,

como a Randomização do Layout do Espaço de Endereçamento (ASLR).

Para otimizar o uso da RAM física e estender a vida útil do armazenamento Flash (eMMC ou UFS), o Android abdica do uso de partições de *swap* tradicionais em disco, as quais gerariam desgaste prematuro do hardware por ciclos excessivos de escrita. Em contrapartida, o sistema adota amplamente o mecanismo ZRAM, que consiste em um bloco de memória comprimido alocado na própria RAM dinâmica. Quando o subsistema de paginação do *kernel* identifica páginas de memória subutilizadas ou pertencentes a processos em segundo plano, essas páginas são compactadas e movidas para a ZRAM de forma síncrona, sendo descompactadas imediatamente quando referenciadas pelo processador, expandindo artificialmente a capacidade utilizável do sistema.

10.2 Mecanismos de controle e otimização de RAM no Android

Devido à natureza altamente interativa dos dispositivos móveis, o Android avalia o consumo real de memória por meio de uma métrica estrita denominada *Proportional Set Size* (PSS). O cálculo do PSS mensura a quantidade de memória RAM física associada a um processo, dividindo proporcionalmente o peso de páginas de bibliotecas e recursos compartilhados entre todos os processos ativos que os utilizam, o que provê uma visão fidedigna do custo real de cada aplicação para o sistema operacional.

Quando os limiares de memória livre atingem níveis críticos que ameaçam a estabilidade da interface gráfica ou das tarefas prioritárias, o Android aciona o *Low Memory Killer Daemon* (*lmkd*) operando no espaço de usuário. O *lmkd* monitora continuamente as flutuações de RAM e, amparado por pontuações dinâmicas de ajuste de falta de memória (*OOM Adjustment scores*), encerra seletivamente os processos menos prioritários. Esse encerramento segue uma hierarquia rígida, liquidando primeiro os aplicativos obsoletos em segundo plano e preservando de forma persistente o processo em foco na tela (*foreground app*), os serviços do sistema e o *framework* gráfico, assegurando baixa latência de transição e alta responsividade visual.

10.3 Abstração de memória no Fuchsia: VMOs e VMARs

No Fuchsia o paradigma de gerenciamento de memória virtual afasta-se por completo dos conceitos clássicos de alocação herdados da tradição Unix/Linux, adotando uma arquitetura puramente orientada a objetos e livre do acoplamento tradicional. O Zircon descentraliza o controle da memória virtual estruturando-o em torno de duas abstrações conceituais fundamentais e complementares: os Objetos de Memória Virtual (VMOs — *Virtual Memory Objects*) e as Regiões de Endereçamento de Memória Virtual (VMARs — *Virtual Memory Address Regions*).

Um VMO representa um contêiner lógico de dados puros que encapsula um conjunto de páginas físicas alocadas ou mapeadas sob demanda. Ele funciona como a representação primordial de memória utilizável no sistema, permitindo operações de leitura, escrita e clonagem via primitivas de comunicação orientadas a objetos. Por outro lado, uma VMAR abstrai o layout e a geometria do próprio espaço de endereçamento virtual atribuído a um processo. As VMARs organizam-se na forma de uma árvore hierárquica, onde a raiz define todo o espaço virtual disponível para o processo e os nós filhos especificam subdivisões protegidas com permissões de execução, leitura e escrita estipuladas rigorosamente em tempo de projeto.

10.4 Segurança por capacidades e isolamento de memória no Zircon

O mapeamento efetivo e o acesso à memória física no Fuchsia estão rigidamente atrelados ao modelo de segurança baseado em capacidades do micronúcleo. Um processo é incapaz de acessar um VMO diretamente; ele deve possuir um *Handle* válido — uma credencial restrita e intransferível emitida pelo *kernel* — que contenha as permissões adequadas para mapear o referido VMO dentro de uma sub-região de sua árvore VMAR correspondente. Esta operação traduz o endereço lógico em tabelas de páginas gerenciadas pelo Zircon, consolidando o controle de acesso direto por hardware.

Visto que os *drivers* de dispositivos no Fuchsia operam inteiramente fora do núcleo, no espaço de usuário, cada subsistema de hardware é alocado em um processo isolado dotado de sua própria árvore hierárquica de VMARs. Diferente do Android, onde uma corrupção de memória ou falha de ponteiro em um *driver* no espaço de *kernel* invalida toda a memória global do sistema, a arquitetura do Fuchsia restringe os impactos a um contexto estritamente local. Caso um componente de hardware sofra um vazamento de memória ou tente acessar um endereço virtual inválido, a violação é contida pela VMAR do processo do *driver*, permitindo que o *kernel* elimine o componente instável e recupere as páginas físicas dos VMOs associados sem afetar a execução dos demais serviços ou do núcleo do sistema.

11 Sistema de entrada e saída e sistemas de arquivos

11.1 Android

O sistema de entrada e saída do Android deriva diretamente do modelo utilizado pelo Linux. As aplicações acessam arquivos e dispositivos utilizando descritores de arquivos (*file descriptors*), enquanto chamadas de sistema encaminham as operações para os respectivos drivers presentes no kernel. Acima dessa camada, o Android disponibiliza diversos frame-

works responsáveis pelo acesso a dispositivos como sensores, câmera, áudio, Bluetooth, rede e demais periféricos utilizados pelas aplicações [2].

O Android oferece suporte a diversos sistemas de arquivos. O principal deles é o `ext4`, amplamente utilizado nas partições internas do sistema. Também há suporte ao `F2FS`, otimizado para memórias Flash, além de `exFAT`, `FAT/VFAT`, `EROFS`, `FUSE`, `IncFS` e sistemas virtuais como `procfs`, `sysfs` e `tmpfs`, utilizados pelo próprio kernel Linux [1].

11.2 Fuchsia

O sistema de entrada e saída do Fuchsia utiliza um modelo baseado em objetos, *handles* e comunicação entre processos (IPC). Em vez de acessar diretamente dispositivos por meio de descritores de arquivos, as aplicações utilizam a biblioteca `fdio`, que fornece uma interface semelhante ao POSIX. As operações são convertidas em mensagens transmitidas por *channels* do Zircon até os serviços responsáveis pelo acesso aos dispositivos, que executam em espaço de usuário. Essa arquitetura proporciona maior isolamento entre componentes e reduz o impacto de falhas em drivers [54, 34].

O Fuchsia suporta diferentes sistemas de arquivos desenvolvidos para finalidades específicas. Entre eles destacam-se o `Fxfs`, considerado o sistema de arquivos moderno da plataforma; o `BlobFS`, utilizado para armazenar pacotes do sistema; o `MinFS`, empregado em componentes persistentes; além de suporte ao `FAT`, `F2FS`, `MemFS` e `FactoryFS`, utilizados conforme as necessidades de cada dispositivo [33].

11.3 Comparação

Embora ambos os sistemas ofereçam suporte a múltiplos sistemas de arquivos e mecanismos modernos de entrada e saída, suas arquiteturas diferem significativamente. O Android herda o modelo tradicional do Linux, baseado em descritores de arquivos e drivers executando no espaço de kernel. Já o Fuchsia adota uma abordagem baseada em objetos e comunicação por mensagens, na qual os drivers e serviços de sistema de arquivos executam predominantemente em espaço de usuário, aumentando o isolamento, a segurança e a modularidade do sistema operacional.

12 Características Arquiteturais Distintivas

12.1 Cadeia de inicialização e o modelo Zygote no Android

Uma característica arquitetural singular do Android que o diferencia de sistemas Linux convencionais é a sua cadeia de inicialização (*boot process*) voltada para a otimização do tempo de abertura de aplicativos. Após o carregamento do *kernel* e a execução do

processo `init` primitivo, o Android inicializa um daemon especial denominado *Zygote*. O *Zygote* é um processo mantido perpetuamente “aquecido” na memória que já contém a máquina virtual Android Runtime (ART), as bibliotecas centrais do *framework* e os recursos gráficos pré-carregados.

Quando o usuário ou o sistema solicita a abertura de um novo aplicativo, em vez de construir um processo do zero, o sistema operacional realiza um `fork()` a partir do processo *Zygote*. Essa abordagem elimina o tempo de inicialização da máquina virtual para cada nova aplicação e permite que todos os aplicativos compartilhem fisicamente as mesmas páginas de memória das bibliotecas base (através do mecanismo *Copy-on-Write*), resultando em uma inicialização quase instantânea e em uma drástica economia de memória global, requisito vital para dispositivos móveis.

12.2 Topologia de inicialização baseada em componentes no Fuchsia

O Fuchsia rejeita o modelo de inicialização linear baseado em *scripts* (como o `init.rc` do Android) e a clonagem de processos monolíticos como o *Zygote*. A inicialização no Fuchsia, após o carregamento da Imagem de Boot do Zircon (ZBI) e do processo inicial `userboot`, é inteiramente delegada ao Gerenciador de Componentes (*Component Manager*). A partir desse ponto, o sistema operacional cresce e se estrutura como uma árvore topológica estrita.

No Fuchsia, não existe a pré-carga de uma máquina virtual central ou global. Como o sistema suporta nativamente múltiplas linguagens (C++, Rust, Dart, Go), cada aplicativo ou serviço do sistema traz o seu próprio ambiente de execução através de um *Runner* específico (por exemplo, um *ELF Runner* para binários nativos ou um *Web Runner* para aplicações web). Os componentes são instanciados do zero, de forma totalmente isolada em seus respectivos contêineres herméticos, priorizando o encapsulamento e a segurança estrutural em detrimento do compartilhamento de estado global.

12.3 Modularidade e a arquitetura Treble/APEX no Android

O ciclo de vida do software do sistema representa outro contraste fundamental. Historicamente prejudicado pela fragmentação de versões, o Android reestruturou sua arquitetura de forma agressiva com os projetos *Treble* e *Mainline*. O *Treble* isolou o código do sistema operacional base do código específico de hardware provido pelas fabricantes através de Camadas de Abstração de Hardware (HAL) rigidamente versionadas, permitindo atualizações do SO sem a necessidade de recompilar *drivers*.

Posteriormente, a introdução do formato de pacote APEX (*Android Pony EXpress*)

permitiu que componentes nativos de baixo nível do sistema (como *codecs* de mídia e resolvedores DNS) fossem atualizados de forma dinâmica e granular. O APEX funciona montando pacotes como sistemas de arquivos em *loopback*, possibilitando que a Google envie atualizações críticas do *kernel* e do sistema diretamente pela loja de aplicativos, desobrigando atualizações completas de *firmware* (OTA) e dependência direta das operadoras de telefonia.

12.4 O paradigma de software efêmero no Fuchsia

O Fuchsia, amparado por sua natureza micronúcleo, leva o conceito de modularidade ao extremo ao abolir a própria ideia de uma "imagem estática" de sistema operacional no espaço de usuário. O Fuchsia adota uma arquitetura radical de “software efêmero”. Nesse modelo, a interface gráfica, os serviços do sistema operacional e os aplicativos de usuário final não são “instalados” no sentido computacional tradicional.

Todo o software executável é tratado como um pacote criptograficamente assinado, identificado e resolvido através de URLs. O sistema baixa, armazena em cache e executa os pacotes estritamente sob demanda. Isso garante que as dependências possam ser atualizadas de forma autônoma e assíncrona, e que o sistema operacional seja inerentemente *evergreen* (sempre em sua versão mais recente). Esse paradigma elimina a necessidade de partições A/B complexas ou reinicializações do dispositivo inteiro apenas para aplicar atualizações a serviços e componentes de interface.

Referências

- [1] Android Open Source Project. Android kernel file system support. <https://source.android.com/docs/core/architecture/android-kernel-file-system-support>. Acesso em: jun. 2026.
- [2] Android Open Source Project. Android open source project. <https://source.android.com/>. Acesso em: jun. 2026.
- [3] Android Open Source Project. Android runtime and dalvik. <https://source.android.com/docs/core/runtime>. Acesso em: jun. 2026.
- [4] Android Open Source Project. Android verified boot. <https://source.android.com/docs/security/features/verifiedboot/avb>. Acesso em: jun. 2026.
- [5] Android Open Source Project. Android virtualization framework (avf) overview. <https://source.android.com/docs/core/virtualization>. Acesso em: jun. 2026.
- [6] Android Open Source Project. App hibernation. <https://source.android.com/docs/core/perf/hiber>. Acesso em: jun. 2026.
- [7] Android Open Source Project. Application sandbox. <https://source.android.com/docs/security/app-sandbox>. Acesso em: jun. 2026.
- [8] Android Open Source Project. Architecture. <https://source.android.com/docs/core/architecture>. Acesso em: jun. 2026.
- [9] Android Open Source Project. Binder ipc. <https://source.android.com/docs/core/architecture/hidl/binder-ipc>. Acesso em: jun. 2026.
- [10] Android Open Source Project. Binder overview. <https://source.android.com/docs/core/architecture/ipc/binder-overview>. Acesso em: jun. 2026.
- [11] Android Open Source Project. Generic kernel image. <https://source.android.com/docs/core/architecture/kernel/generic-kernel-image>. Acesso em: jun. 2026.
- [12] Android Open Source Project. Input. <https://source.android.com/docs/core/interaction/input>. Acesso em: jun. 2026.
- [13] Android Open Source Project. Kernel overview. <https://source.android.com/docs/core/architecture/kernel>. Acesso em: jun. 2026.
- [14] Android Open Source Project. Low memory killer daemon. <https://source.android.com/docs/core/perf/lmkd>. Acesso em: jun. 2026.

- [15] Android Open Source Project. Mainline. <https://source.android.com/docs/core/ota/modular-system>. Acesso em: jun. 2026.
- [16] Android Open Source Project. Performance. <https://source.android.com/docs/core/perf>. Acesso em: jun. 2026.
- [17] Android Open Source Project. Platform power management. https://source.android.com/docs/core/power/platform_mgmt. Acesso em: jun. 2026.
- [18] Android Open Source Project. Security-enhanced linux in android. <https://source.android.com/docs/security/features/selinux>. Acesso em: jun. 2026.
- [19] Android Open Source Project. Systemsuspend service. <https://source.android.com/docs/core/power/systemsuspend>. Acesso em: jun. 2026.
- [20] Android Open Source Project. Thermal mitigation. <https://source.android.com/docs/core/power/thermal-mitigation>. Acesso em: jun. 2026.
- [21] Android Open Source Project. Zygote. <https://source.android.com/docs/core/runtime/zygote>. Acesso em: jun. 2026.
- [22] Google. Android. <https://www.android.com/>. Acesso em: jun. 2026.
- [23] Google. Android 16. <https://developer.android.com/about/versions/16>. Acesso em: jun. 2026.
- [24] Google. Android 17 release notes. <https://developer.android.com/about/versions/17/release-notes>. Acesso em: jun. 2026.
- [25] Google. Build fuchsia. https://fuchsia.dev/fuchsia-src/get-started/build_fuchsia. Acesso em: jun. 2026.
- [26] Google. Channel. https://fuchsia.dev/fuchsia-src/reference/kernel_objects/channel. Acesso em: jun. 2026.
- [27] Google. Choreographer. <https://developer.android.com/reference/android/view/Choreographer>. Acesso em: jun. 2026.
- [28] Google. Diagnose and fix anrs. <https://developer.android.com/topic/performance/anrs/diagnose-and-fix-anrs>. Acesso em: jun. 2026.
- [29] Google. Fuchsia. <https://fuchsia.dev/>. Acesso em: jun. 2026.
- [30] Google. Fuchsia component capabilities. <https://fuchsia.dev/fuchsia-src/concepts/components/v2/capabilities>. Acesso em: jun. 2026.

- [31] Google. Fuchsia principle: Secure. <https://fuchsia.dev/fuchsia-src/concepts/principles/secure>. Acesso em: jun. 2026.
- [32] Google. Fuchsia principle: Updatable. <https://fuchsia.dev/fuchsia-src/concepts/principles/updatable>. Acesso em: jun. 2026.
- [33] Google. Fuchsia storage and file systems. <https://fuchsia.googlesource.com/fuchsia/+refs/heads/main/src/storage/fxfs/README.md>. Acesso em: jun. 2026.
- [34] Google. fuchsia.io. <https://fuchsia.dev/reference/fidl/fuchsia.io>. Acesso em: jun. 2026.
- [35] Google. Futex. https://fuchsia.dev/fuchsia-src/reference/kernel_objects/futex. Acesso em: jun. 2026.
- [36] Google. Googlebook. <https://developer.android.com/googlebook>. Acesso em: jun. 2026.
- [37] Google. Improve app performance with kotlin coroutines. <https://developer.android.com/kotlin/coroutines/coroutines-adv>. Acesso em: jun. 2026.
- [38] Google. Introduction to components. <https://fuchsia.dev/fuchsia-src/concepts/components/v2/introduction>. Acesso em: jun. 2026.
- [39] Google. Introduction to fuchsia components. <https://fuchsia.dev/fuchsia-src/concepts/components/v2/introduction>. Acesso em: jun. 2026.
- [40] Google. Introduction to zircon. <https://fuchsia.dev/fuchsia-src/get-started/learn/intro/zircon>. Acesso em: jun. 2026.
- [41] Google. Object. <https://developer.android.com/reference/java/lang/Object>. Acesso em: jun. 2026.
- [42] Google. Optimize for doze and app standby. <https://developer.android.com/topic/performance/appstandby>. Acesso em: jun. 2026.
- [43] Google. Overview of memory management. <https://developer.android.com/topic/performance/memory-management>. Acesso em: jun. 2026.
- [44] Google. Packages. <https://fuchsia.dev/fuchsia-src/get-started/learn/intro/packages>. Acesso em: jun. 2026.
- [45] Google. Platform versioning compatibility. <https://fuchsia.dev/fuchsia-src/concepts/versioning/compatibility>. Acesso em: jun. 2026.

- [46] Google. Processes and app lifecycle. <https://developer.android.com/guide/components/activities/process-lifecycle>. Acesso em: jun. 2026.
- [47] Google. Processes and threads overview. <https://developer.android.com/guide/components/processes-and-threads>. Acesso em: jun. 2026.
- [48] Google. Sandboxing. <https://fuchsia.dev/fuchsia-src/get-started/learn/intro/sandboxing>. Acesso em: jun. 2026.
- [49] Google. Socket. https://fuchsia.dev/fuchsia-src/reference/kernel_objects/socket. Acesso em: jun. 2026.
- [50] Google. Starnix. <https://fuchsia.dev/fuchsia-src/concepts/starnix>. Acesso em: jun. 2026.
- [51] Google. Starnix kernel. <https://fuchsia.dev/fuchsia-src/concepts/starnix/kernel>. Acesso em: jun. 2026.
- [52] Google. Thread. https://fuchsia.dev/fuchsia-src/reference/kernel_objects/thread. Acesso em: jun. 2026.
- [53] Google. Zircon kernel concepts. <https://fuchsia.dev/fuchsia-src/concepts/kernel/concepts>. Acesso em: jun. 2026.
- [54] Google. Zircon kernel documentation. <https://fuchsia.dev/fuchsia-src/concepts/kernel>. Acesso em: jun. 2026.
- [55] StatCounter Global Stats. Operating system market share worldwide. <https://gs.statcounter.com/os-market-share>. Acesso em: jun. 2026.
- [56] Andrew S. Tanenbaum and Herbert Bos. *Modern Operating Systems*. Pearson, 4 edition, 2015.
- [57] Wikipedia. Android (operating system). [https://en.wikipedia.org/wiki/Android_\(operating_system\)](https://en.wikipedia.org/wiki/Android_(operating_system)). Acesso em: jun. 2026.
- [58] Wikipedia. Fuchsia (operating system). [https://en.wikipedia.org/wiki/Fuchsia_\(operating_system\)](https://en.wikipedia.org/wiki/Fuchsia_(operating_system)). Acesso em: jun. 2026.
- [59] Keun Soo Yim, Iliyan Malchev, Andrew Hsieh, and Dave Burke. Treble: Fast software updates by creating an equilibrium in an active software ecosystem of globally distributed stakeholders. *arXiv preprint arXiv:2410.02809*, 2024. Acesso em: jun. 2026.